DM码识别

DM码识别
- 例程效果简介
  - 什么是DM码
  - DM码和 QRCode（二维码）的区别
  - 生成DM码
  - 运行例程
- 代码讲解
  - 关键代码
  - 代码结构

例程效果简介

什么是DM码

DM码（Data Matrix码）是一种二维条码，有以下特点：

1. 形状为正方形或长方形，由黑白点阵组成
2. 支持较小尺寸，适合小面积标识
3. 抗损坏能力强，即使破损30%仍可读取
4. 编码密度高，可存储更多数据

DM码和 QRCode（二维码）的区别

1. 外观识别：

- DM码：只有实线边框
- QR码：有3个定位点标识

2. 尺寸：

- DM码：可做得很小，适合微小产品
- QR码：有最小尺寸限制，较大

3. 应用场景：

- DM码：工业标识、小型零件追溯
- QR码：营销、支付、网址分享等大众应用

4. 存储容量：

- DM码：最多2335个字母数字
- QR码：最多4296个字母数字

生成DM码

可以使用我们资料中提供的DM码生成工具，也可以在网上搜索在线DM生成器使用



运行例程

连接IDE后点击左下角运行按钮，用K230对准我们刚刚生成的DM码，可以看到屏幕上识别出了DM码的内容

右边中括号里的[1.00]指的是DM码旋转偏移的角度



> 需要注意的是，DM的识别成功率比QRCode二维码低，如果一直无法识别，可以尝试旋转DM码或者摄像头

代码讲解

本节完整例程代码文件位于 [源码汇总 / 06.Codes / 04.find_datamatrices]

关键代码

```
# 使用默认配置构造 Sensor 对象
# Create a Sensor object with default configuration
sensor = Sensor()
# 重置 sensor
# Reset the sensor
sensor.reset()

# 设置输出大小和格式
# Set the output size and pixel format
sensor.set_framesize(width=DETECT_WIDTH, height=DETECT_HEIGHT)
sensor.set_pixformat(Sensor.RGB565)
#    sensor.set_pixformat(Sensor.GRAYSCALE)


# 初始化显示
# Initialize the display
Display.init(Display.ST7701, width=DETECT_WIDTH, height=DETECT_HEIGHT,
to_ide=True)

# 初始化媒体管理器
# Initialize the media manager
MediaManager.init()
```

```
# 启动 sensor
# Start the sensor
sensor.run()

fps = time.clock()

while True:
    fps.tick()

    # 检查是否应该退出
    # Check if we should exit
    if os.exitpoint():
        break

        img = sensor.snapshot()

        # 遍历图像中的 Data Matrix 条形码
        # Iterate through the Data Matrix codes found in the image
        for matrix in img.find_datamatrices():
            # 绘制识别到的 Data Matrix 码的矩形框
            # Draw the rectangle around the detected Data Matrix code
            (x, y, w, h) = matrix.rect()
            y = y - 25 if y - 25 > 0 else y
            img.draw_rectangle([v for v in matrix.rect()], color=(255, 0, 0),
thickness=4)
            # 打印矩阵的行列数、内容、旋转角度（度）以及当前的 FPS
            # Print the matrix's row, column count, payload, rotation in
degrees, and current FPS
            print_args = (matrix.payload(), (180 * matrix.rotation()) / math.pi)
            img.draw_string_advanced(x, y, 20, "%s [%.2f]°" % print_args, color=
(255,0,0))

            print("内容 payload \"%s\", 旋转 rotation %f (度)" % print_args)

            # 将结果显示到屏幕上
            # Display the result on the screen
            Display.show_image(img)

            # 进行垃圾回收，释放内存
            # Perform garbage collection to release memory
            gc.collect()
```

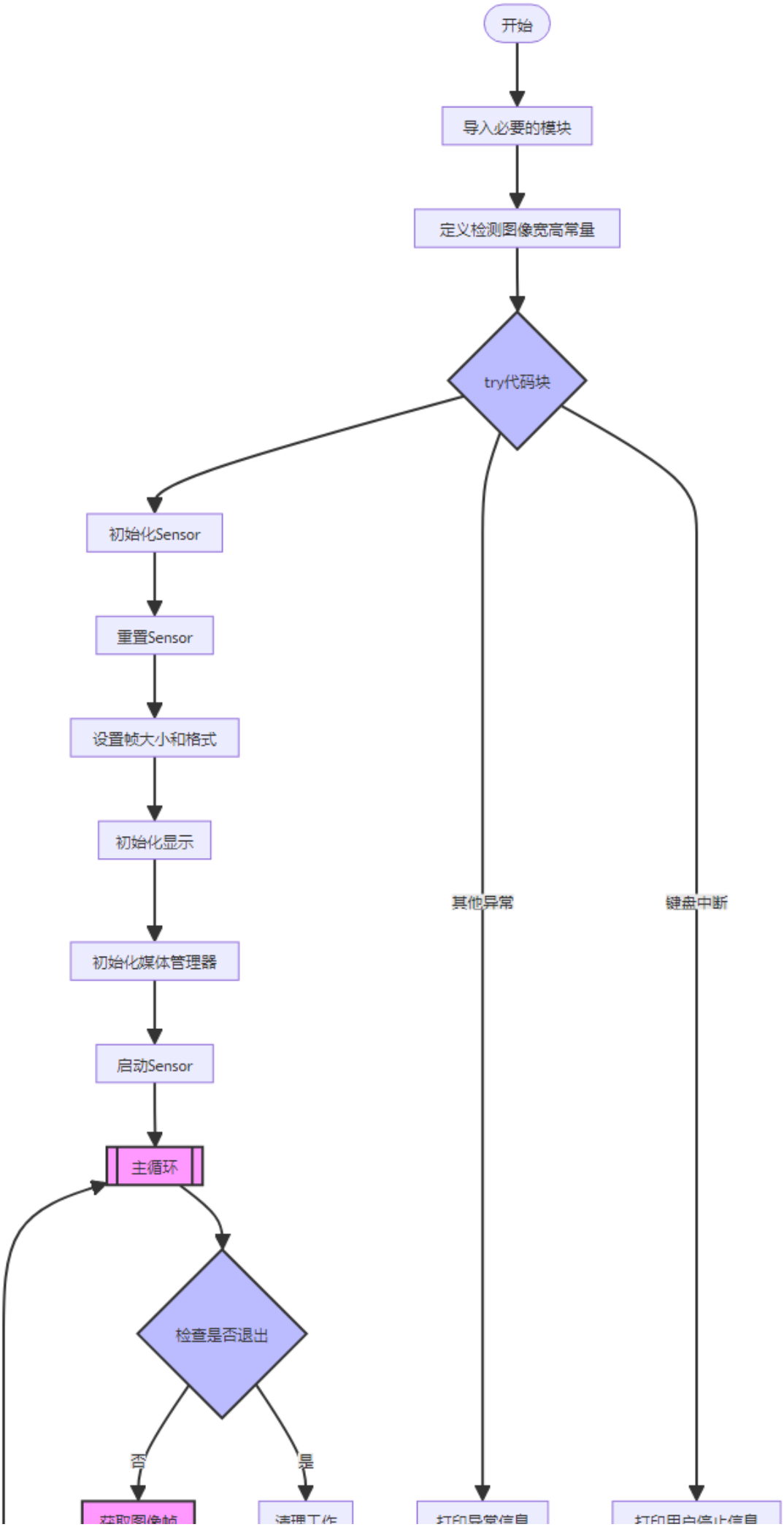
代码结构

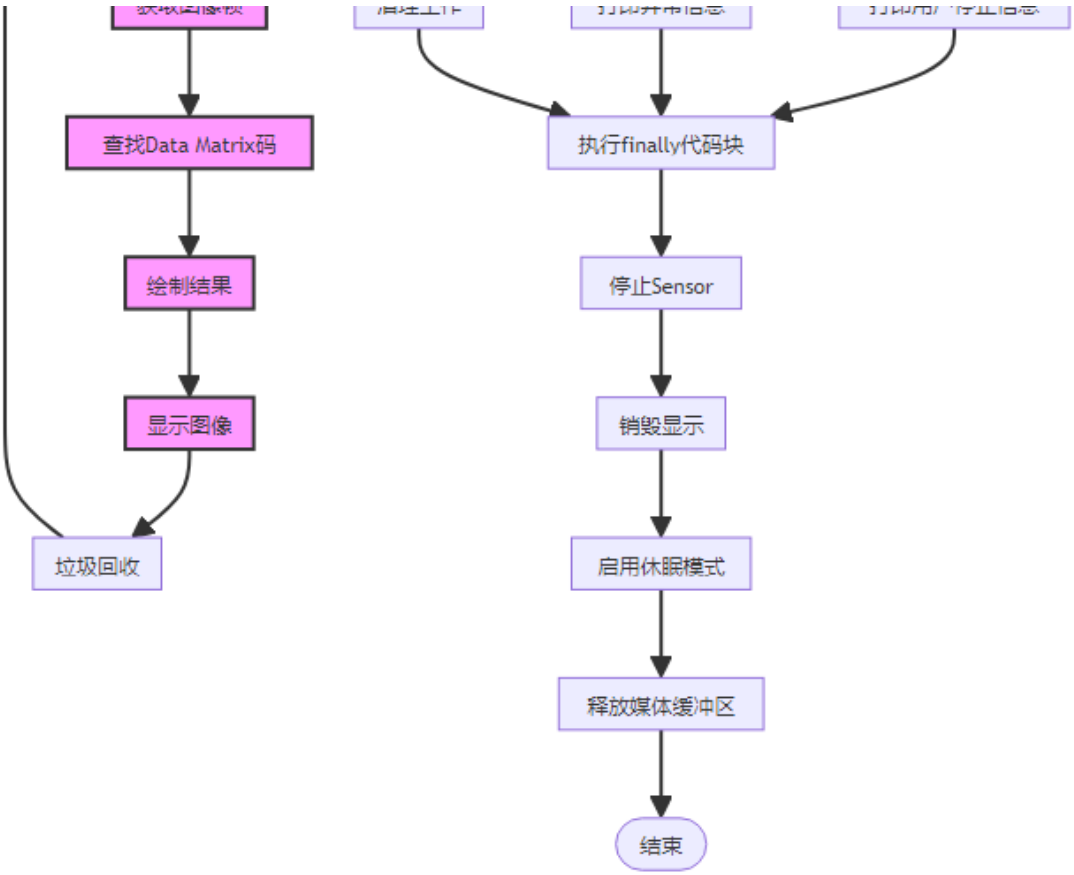
执行流程：

1. 首先导入必要的模块，定义常量
2. 在try代码块中:
    - 初始化和配置Sensor
    - 初始化显示和媒体管理器
    - 启动Sensor
    - 进入主循环:
        - 检查是否需要退出

        - 获取图像帧
        - 查找Data Matrix码
        - 绘制和显示结果
        - 执行垃圾回收
3. 异常处理:
    - 处理键盘中断
    - 处理其他异常
4. 最后在finally块中:
    - 停止Sensor
    - 销毁显示
    - 启用休眠模式
    - 释放媒体缓冲区

详细流程图如下:


开始
导入必要的模块
定义检测图像宽高常量
try代码块
初始化Sensor
重置Sensor
设置帧大小和格式
初始化显示
初始化媒体管理器
启动Sensor
主循环
检查是否退出
否
是
其他异常
键盘中断


查找Data Matrix码
绘制结果
显示图像
垃圾回收
执行finally代码块
停止Sensor
销毁显示
启用休眠模式
释放媒体缓冲区
结束